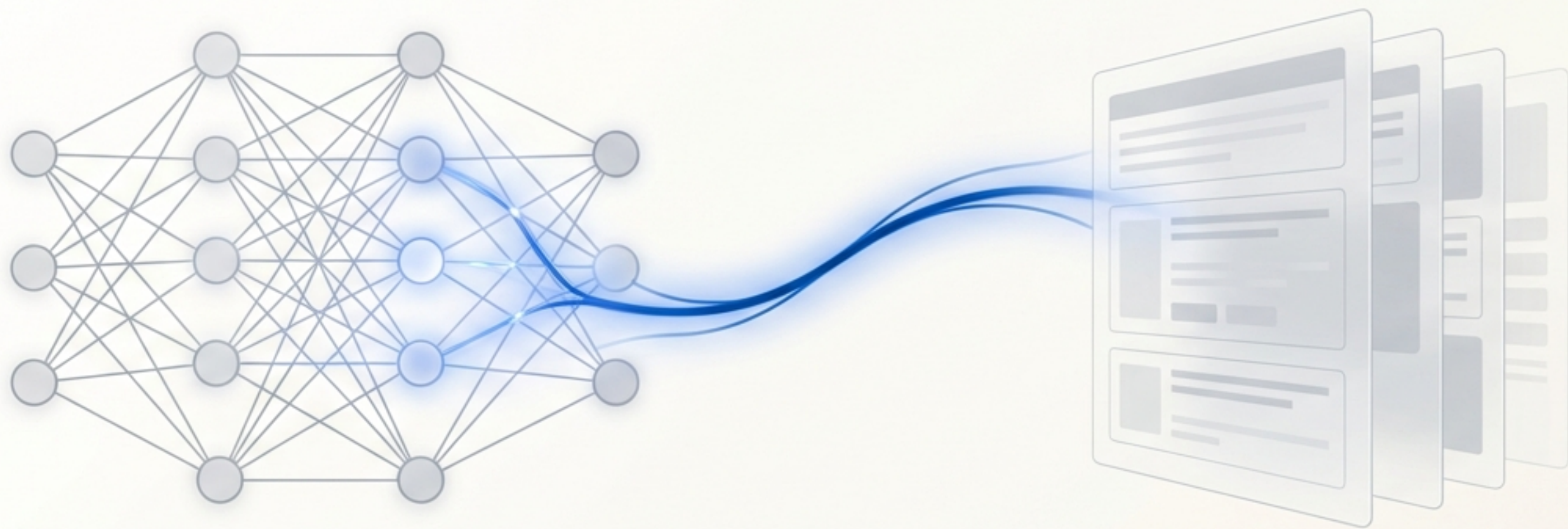




RAG：让大模型拥有实时记忆和专业知识

检索增强生成技术深度解析与实战指南

大模型的“知识截止日期”困境

模型训练数据（截止于 2023年）

持续演变的真实世界



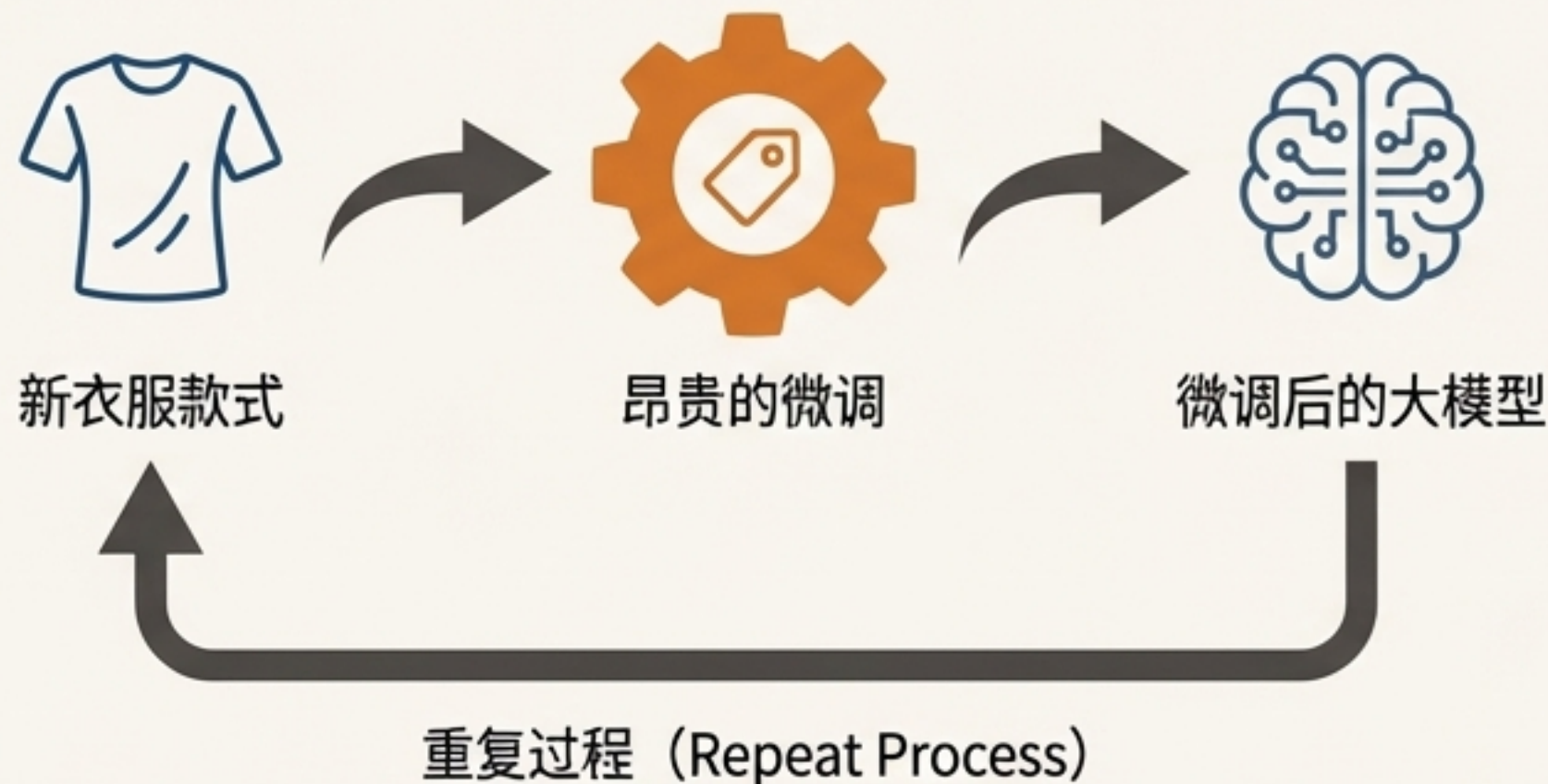
- 大语言模型（LLM）基于庞大但静态的数据集进行训练，其知识被“冻结”在训练完成的那个时刻。
- 它们无法获知任何私有的、专有的或实时更新的信息，例如：公司的内部规章、最新的产品库存、今天的财经新闻。
- 这导致了两个核心问题：一是事实性错误或“幻觉”；二是对特定领域或新知识的“一无所知”。

为什么“微调”并非万能解药？

Core Argument

微调擅长教会模型新的技能或对话风格，但在更新知识方面存在明显短板。

“你想想，总不能说我进了一款衣服，就做微调，做微调。”



高昂成本 (High Cost) :

需要大量的标注数据和昂贵的GPU算力进行重复训练。



迟缓与僵化 (Slow & Inflexible) :

无法为每一份更新的文档或每一条新数据都重新训练模型。对于电商库存、企业知识库这类动态信息，微调不切实际。



知识遗忘 (Forgetting Risk) :

微调有时会损害模型原有的通用能力。

解决方案 RAG：为大模型外挂一个“实时知识大脑”

核态理念：与其将知识强行“塞入”模型（微调），不如让模型在需要时主动从外部知识源中“检索”信息。



大语言模型



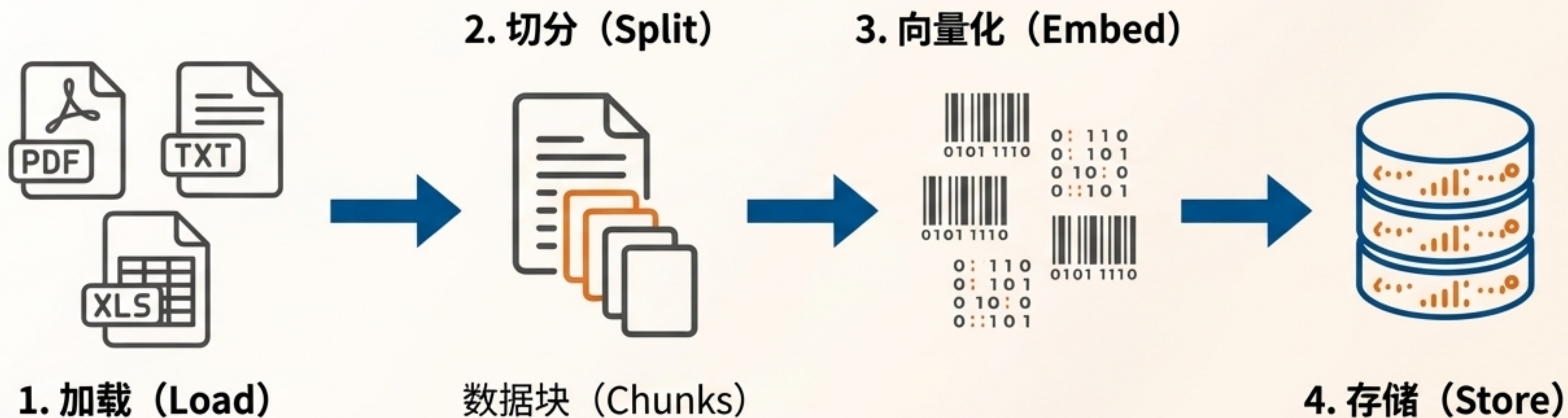
向量数据库



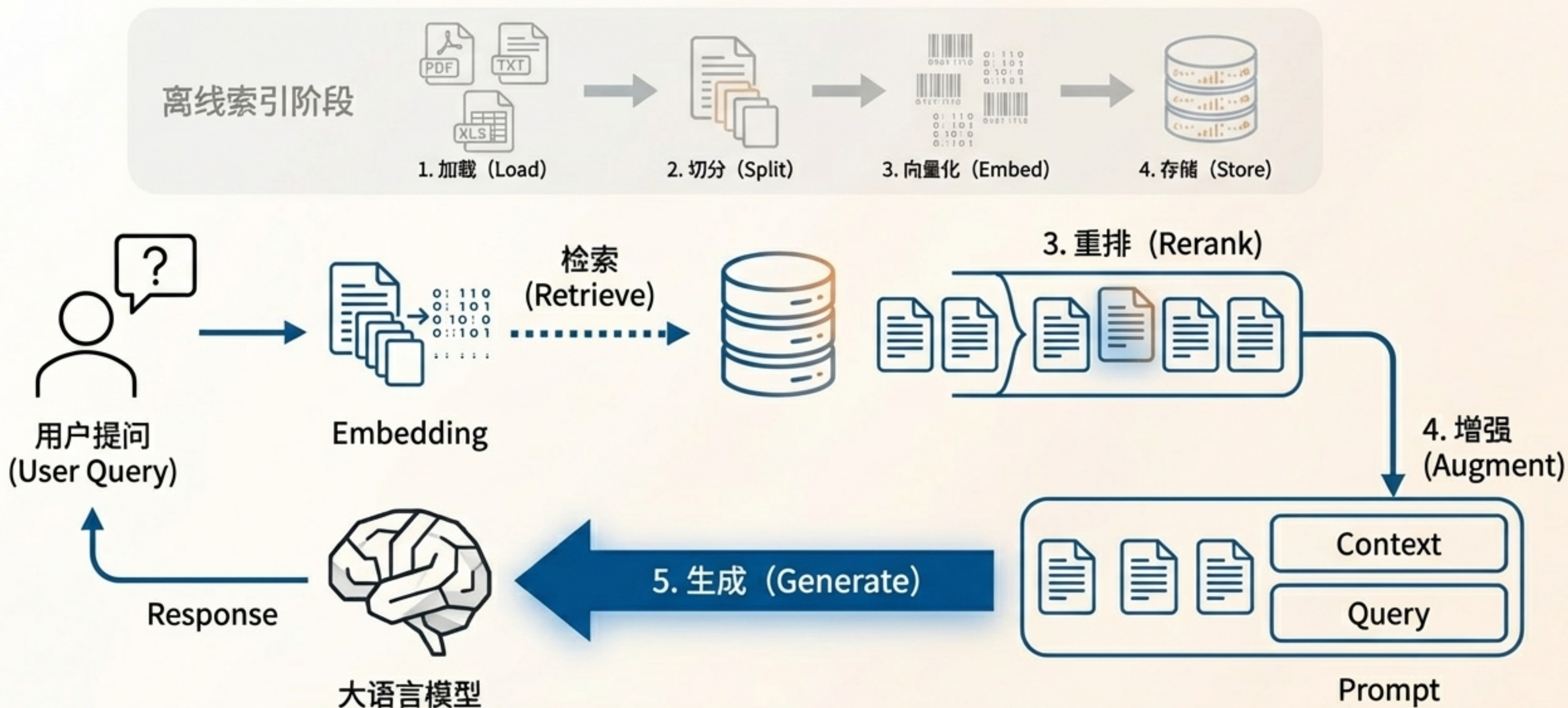
更新知识库变得轻而易举。当信息变化时（例如，衣服上新或下架），我们只需操作向量数据库（insert/delete），而无需触碰大模型本身。

分解：这就像让模型从一个“闭卷考生”变成一个“开卷宗师”。它不需要记住图书馆里的每一本书，只需要知道如何快速查阅索引。

RAG 架构 Part 1: 构建可检索的知识库 (索引阶段)



RAG 架构 Part 2: 实时问答的“检索-增强-生成”流程



RAG 实战对比：从泛泛而谈到精准回答

用户提问：“公司的聘用原则是什么？”

WITHOUT RAG

LLM:

聘用原则通常指的是在雇佣关系中，雇主和雇员之间应该遵循的一些基本原则...

1. 公平原则...
2. 透明原则...
3. 合法原则...

✗ 评价：正确但毫无用处。

WITH RAG



知识源

`汇视威管理制度.txt`

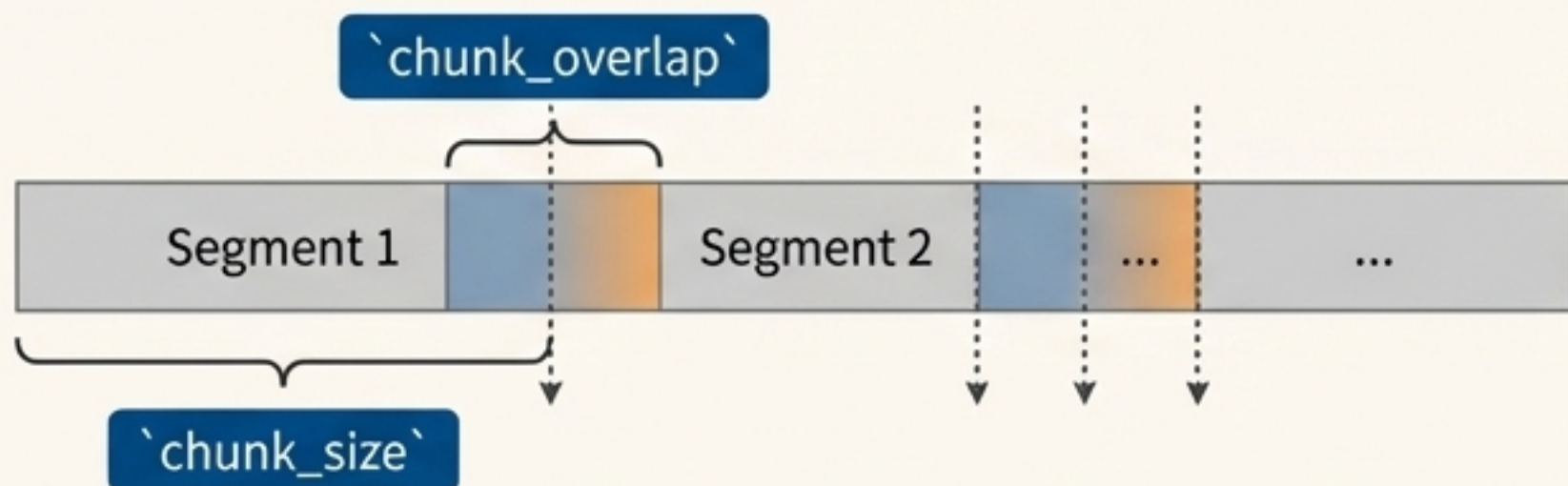
公司的聘用原则分为**基础岗位人员任用**和**管理岗位人员任用**。

1. 对于基础岗位人员，由用人部门主管面试确定。
2. 对于管理岗位人员，由总经理最终面试确定。

注：基层岗位主管岗位以下人员均属于基础岗位。

✓ 评价：精准、可溯源、有价值。

关键细节：文本切分的科学与艺术



```
self.text_splitter = RecursiveCharacterTextSplitter(  
    separators=["\n\n", "\n", " ", ""],  
    chunk_size=1000,  
    chunk_overlap=200,  
    length_function=len,  
)
```

Why Split Text?

- **模型上下文限制**：LLM的输入窗口（Context Window）大小有限。
- **检索精度**：更小的、语义集中的文本块能提供更精确的检索结果。

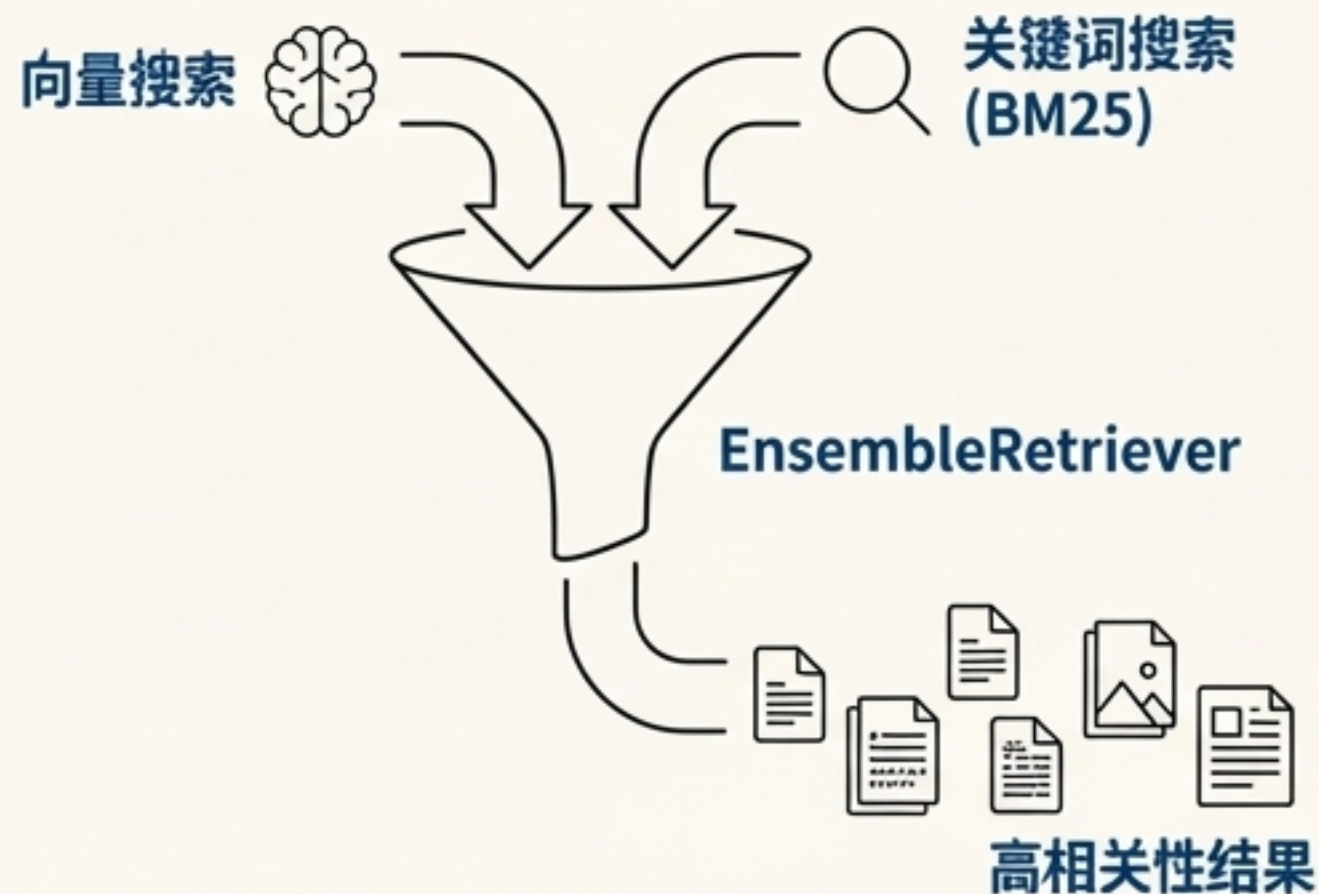
Key Parameters Explained

- **'chunk_size'**：每个数据块的大小。例如，1000个字符。决定了单次检索返回内容的信息量。
- **'chunk_overlap'**：数据块之间的重叠大小。例如，200个字符。这能有效防止在切分边界丢失完整的语义信息，确保数据的连续性。

超越简单搜索：混合检索（Hybrid Search）提升召回率

The Problem with a Single Method

- **向量搜索**（Vector Search）：擅长理解语义和概念（“招聘相关的政策”），但可能忽略特定的关键词或产品型号。
- **关键词搜索**（Keyword Search, e.g., BM25）：精准匹配关键词和缩写，但无法理解语义上的相似性。



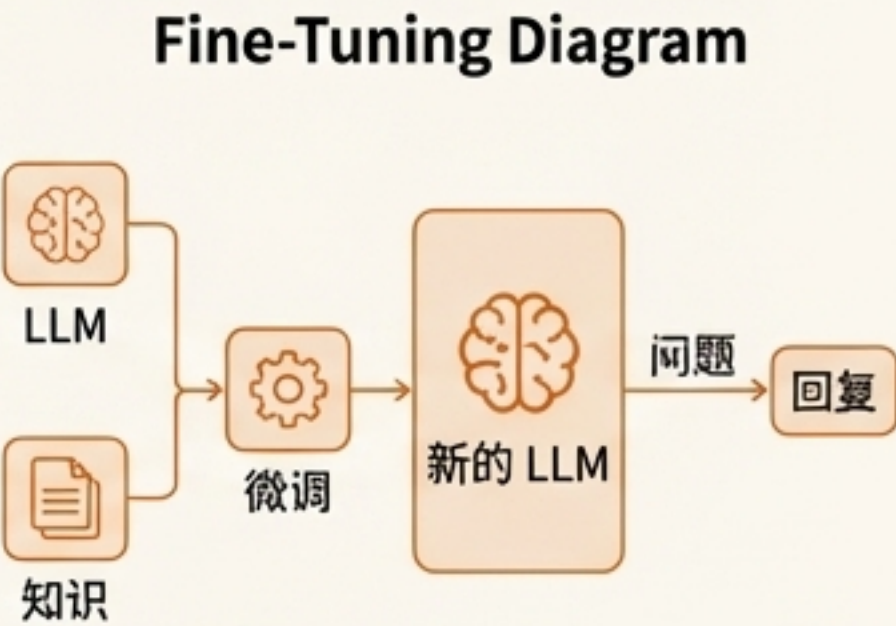
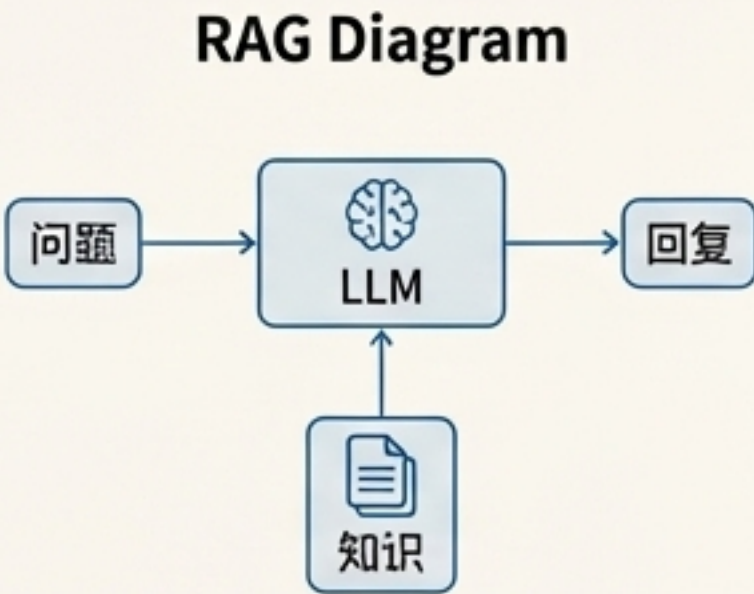
The Solution: Ensemble Retriever

结合多种检索器的优势，确保既能找到语义相关的内容，又不会错过包含精确关键词的文档。

```
# 混合检索，将稀疏检索器（如 BM25）与密集检索器（如嵌入相似性）相结合
ensemble_retriever = EnsembleRetriever(
    retrievers=[db.as_retriever(search_kwargs={"k": 3}),
                BM25Retriever.from_documents(documents)]
)
```


如何抉择？ RAG 与微调的适用场景

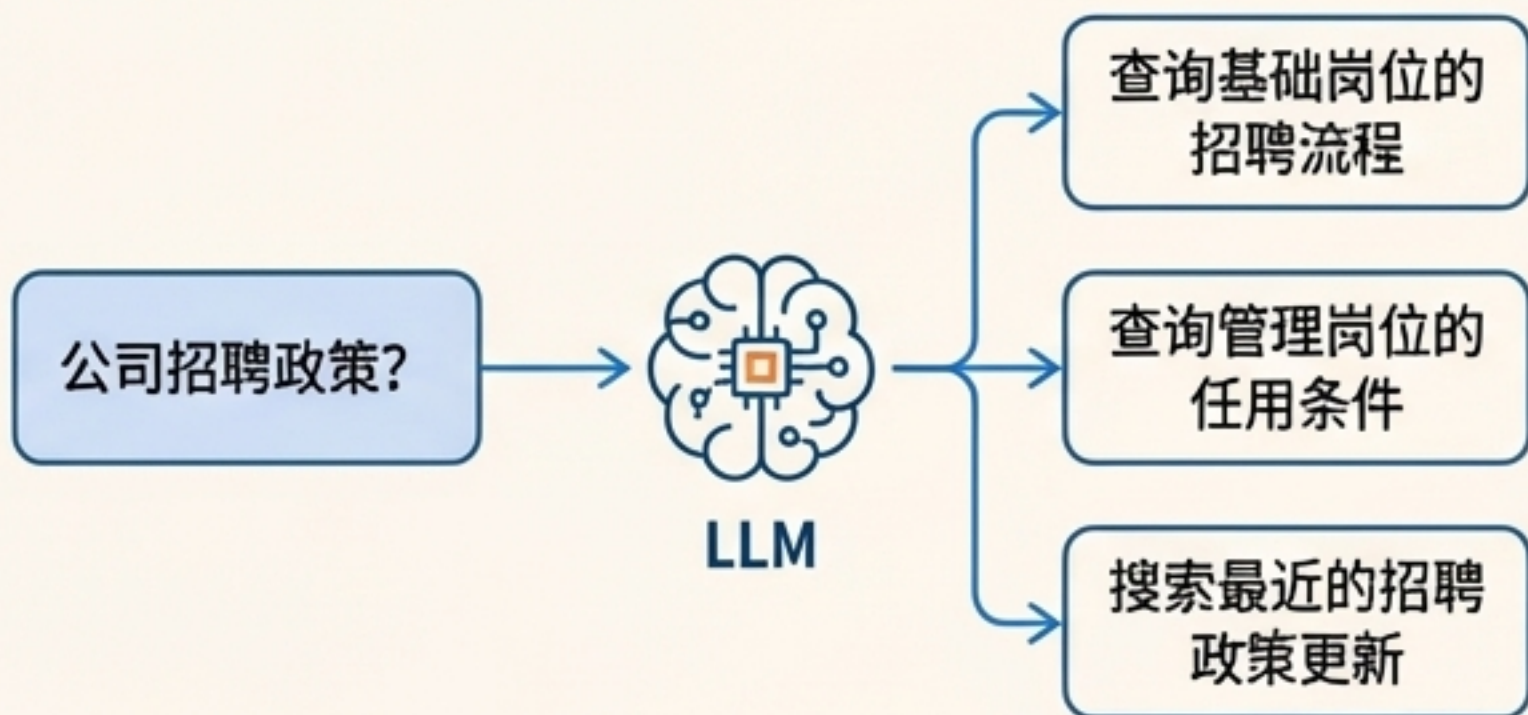
场景维度	推荐方案	核心理由
动态数据/实时知识	RAG	知识库可随时增删改查，成本极低，无需重新训练模型。
模型能力定制	微调	旨在改变模型的内在行为、风格或掌握新技能。
对抗模型幻觉	RAG > 微调	答案基于外部提供的确凿证据，有据可查，可溯源。
可解释性	RAG	可以清晰展示用于生成答案的原始文本片段。
成本	RAG	避免了昂贵的模型训练和部署成本，迭代速度快。
依赖通用能力	RAG	充分利用基础模型的强大通用能力，仅外挂知识。
延迟	微调	RAG 增加了检索步骤，会引入额外的毫秒级延迟。



RAG 的未来：从简单检索到深度理解

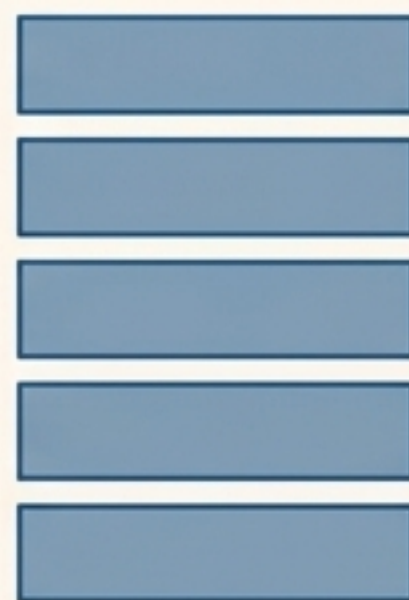
1. 查询转换 (Query Transformation)

在检索前，先利用LLM优化、重写甚至分解用户的原始问题，以生成更适合在知识库中检索的查询。例如，“用户问一个问题，先让大模型把这个问从不同角度生成多个问题”。This improves retrieval accuracy.

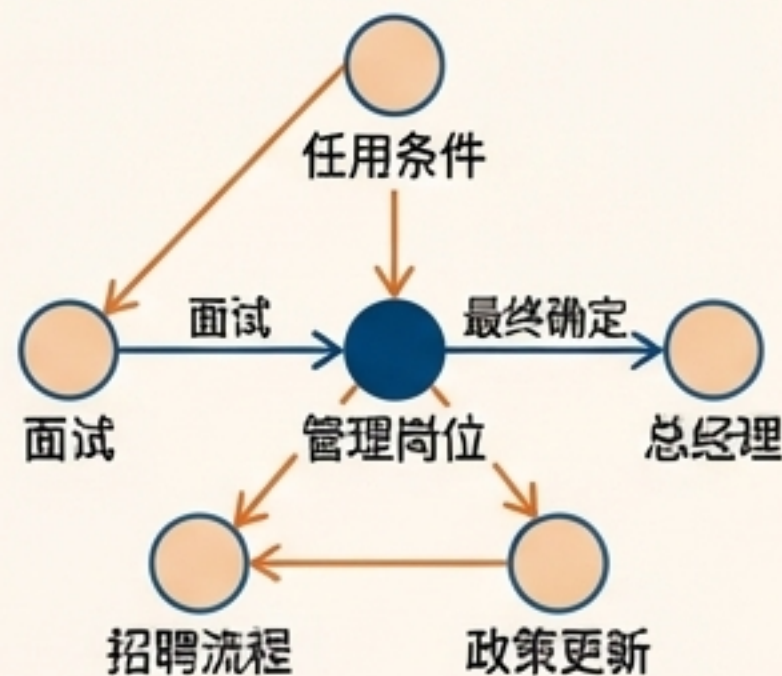


2. GraphRAG

使用知识图谱 (Knowledge Graphs) 替代纯文本块作为知识源。这使得模型不仅能检索信息，还能理解和推理实体之间的复杂关系。一个值得关注的实践是微软的开源项目 GraphRAG。



Text Chunks

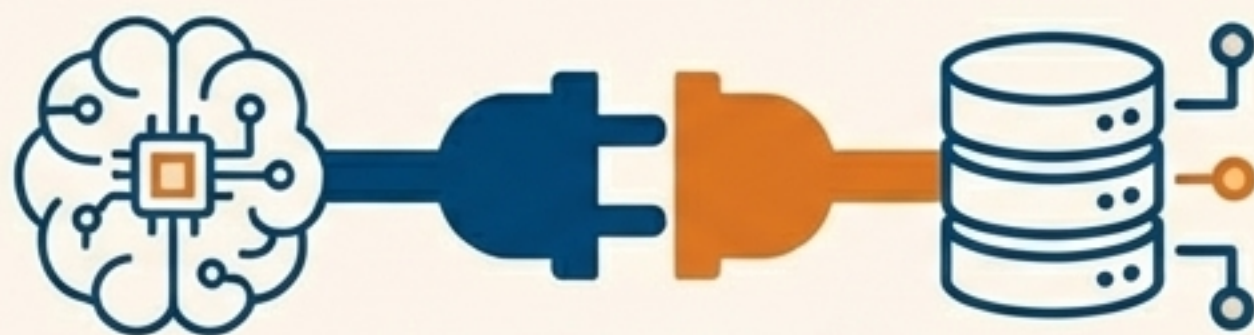


Knowledge Graph

核心要点回顾：打造更智能、更可靠的AI



RAG 弥合知识鸿沟：它将LLM与动态、私有的外部数据源连接起来，从根本上解决了模型的知识时效性与幻觉问题。



兼具灵活性与成本效益：对于知识密集型任务，RAG通常比微调更经济、更易于维护和扩展。



细节决定成败：一个成功的RAG系统的效果，高度依赖于在文本切分(Chunking)、向量化(Embedding)和检索策略(Retrieval)上的明智选择。

The background features a light beige circular gradient. Overlaid on this are three faint, stylized icons: a brain with a central microchip on the left, a hand in the center, and a database cylinder with connecting lines on the right.

Q&A

Thank You